

QUEUE

What is a Queue?

A Queue is a linear data structure that follows the FIFO (First In, First Out) principle.

This means the first element inserted into the queue is the first one to be removed.

Think of a queue like people standing in a line:

- A person joins at the rear end
- A person leaves from the front end first

Unlike stacks, queues allow insertion and deletion at different ends:

- Insertion happens at the **Rear**
- Deletion happens at the **Front**

Basic Operations of Queue

1. **Enqueue** – Insert an element into the queue
2. **Dequeue** – Remove an element from the queue
3. **Front (Peek)** – View the front element without removing it
4. **isEmpty** – Check if the queue is empty
5. **isFull** – Check if the queue is full (in array implementation)

Representation of Queue

Queue can be implemented in two ways:

- Using Arrays
- Using Linked Lists

Example (Array Representation):

Front → [10] [20] [30] ← Rear

After inserting 10, 20, 30:

- Front points to 10
- Rear points to 30

Queue Using Array in C

```
#define MAX 100
```

```
int queue[MAX];  
int front = -1;  
int rear = -1;
```

Explanation:

- `queue[MAX]` defines an array to store elements
- `front` keeps track of the first element
- `rear` keeps track of the last element
- Initially, both are `-1`, meaning the queue is empty

Queue Operations in C

Enqueue Operation

```
void enqueue(int x) {  
    if (rear == MAX - 1) {  
        printf("Queue Overflow\n");  
    } else {  
        if (front == -1)  
            front = 0;  
  
        rear++;  
        queue[rear] = x;  
    }  
}
```

Dequeue Operation

```
void dequeue() {  
    if (front == -1 || front > rear) {  
        printf("Queue Underflow\n");  
    } else {  
        printf("Deleted element: %d\n", queue[front]);  
        front++;  
  
        if (front > rear) {  
            front = rear = -1;  
        }  
    }  
}
```

Front (Peek) Operation

```
void peek() {
    if (front == -1) {
        printf("Queue is empty\n");
    } else {
        printf("Front element: %d\n", queue[front]);
    }
}
```

Queue Using Linked List

Structure of Node:

```
struct Node {
    int data;
    struct Node* next;
};
```

- Each node stores data and a pointer to the next node
- `front` points to the first node
- `rear` points to the last node

Characteristics of Queue

- Follows FIFO principle
- Insertion happens at rear and deletion happens at front
- Can be implemented using arrays or linked lists
- Efficient for scheduling and resource sharing

Advantages of Queue

1. Fair Processing

Elements are processed in the order they arrive

2. Efficient Operations

Enqueue and dequeue operations take constant time $O(1)$

3. Useful in Scheduling

Widely used in CPU and printer scheduling

4. Dynamic Memory Management (Linked List)

Queue size can grow dynamically

Disadvantages of Queue

1. Limited Size (Array Implementation)

Queue overflow may occur

2. Memory Wastage

Unused spaces may remain in simple array queues

3. No Random Access

Elements cannot be directly accessed

Real-Life Use Cases of Queue

1. CPU Scheduling

Processes are executed in arrival order

2. Printer Queue

Print jobs are processed one by one

3. Ticket Booking Systems

Customers are served in sequence

4. Breadth First Search (BFS)

Queues are used in graph traversal

5. Network Packet Handling

Data packets are processed in order

6. Call Center Systems

Calls are handled on a first-come-first-serve basis

Types of Queue

1. Simple Queue

Insertion at rear and deletion at front

2. Circular Queue

Last position connects back to the first position

3. Priority Queue

Elements are processed based on priority

4. Double Ended Queue (Deque)

Insertion and deletion can happen at both ends

Key Points to Remember

- Queue follows First In First Out (FIFO)
- Insertion is called enqueue
- Deletion is called dequeue
- Front points to the first element
- Rear points to the last element
- Overflow occurs when queue is full
- Underflow occurs when queue is empty
- Widely used in scheduling, buffering, and graph traversal